

ISW - Tutorato

s.dessi63@studenti.unica.it

l.piras78@studenti.unica.it

OGGETTO MAIL "ISW"

Cosa vedremo oggi

- ▶ Overriding
- ▶ File
- ▶ Args – Kwargs

Programmazione OO: Override

- ▶ Il meccanismo di eredità dei metodi è simile a quello del C++ e del Java:
- ▶ Tutti i metodi di una classe base sono ereditati dalla classe figlia.
- ▶ I metodi statici e di classe si ereditano con le stesse regole dei metodi di Istanza.
- ▶ Se ridefinisco un metodo, la chiamata al relativo metodo della classe base non è automatica.

Programmazione OO: Metodi Speciali

- ▶ Il meccanismo di eredità dei metodi è simile a quello del C++ e del Java:
- ▶ Tutti gli attributi/metodi che iniziano e finiscono con `__` hanno un significato speciale in Python
- ▶ Alcuni li abbiamo già visti:
 - ▶ `__init__`
- ▶ Vediamo:
 - ▶ `__repr__`
 - ▶ `__str__`
 - ▶ `__class__`
 - ▶ `__name__`

Programmazione OO: Metodi Speciali

- ▶ **repr** serve a costruire una rappresentazione “completa” dell'oggetto.
 - ▶ La stringa generata deve permettere di ricostruire l'oggetto.
- ▶ **str** serve a costruire una rappresentazione “informale” dell'oggetto.
 - ▶ La stringa generata dovrebbe essere facilmente leggibile per l'utente finale, quindi più semplice e descrittiva.
 - ▶ **repr** è più per sviluppatori mentre **str** è più per utente finale.

Programmazione OO: Metodi Speciali

- ▶ **`__repr__`** serve a costruire una rappresentazione “completa” dell'oggetto.

```
class P:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return 'x: %d, y: %d' %(self.x, self.y)
    def __repr__(self):
        return 'P(%d, %d)' %(self.x, self.y)
...
p1 = P(1, 3)
print(p1) #viene chiamata __str__
p1 #su console viene visualizzata la stringa restituita da __repr__
```

Programmazione OO: Metodi Speciali

- ▶ **`__class__`** è un attributo che contiene un riferimento alla classe a cui appartiene l'oggetto.
- ▶ Tutti gli oggetti, comprese le classi, hanno l'attributo **`__class__`**

```
class Foo:
```

```
    pass
```

```
obj = Foo()
```

```
print(obj.__class__) #riferimento
```

Programmazione OO: Metodi Speciali

- ▶ **__name__** è' un attributo che contiene il nome di una classe, di un modulo o di un package.
Tutte le classi, comprese le *built-in*, hanno l'attributo **__name__**

```
print(obj. __class__. __name__) #“Foo”  
print(int. __name__) #“int”  
print(list. __name__) #“list”  
print(list. __class__. __name__) #“type”
```


Python - File

- ▶ Elaborazione di dati molto grandi sia in input sia per salvarli
- ▶ Diversi tipi di file:
 - ▶ txt: semplici file di testo
 - ▶ json: formato per interpretazione dati (client-server)
 - ▶ csv: elaborazione dati in tabella
 - ▶ pickle: oggetto python in flusso di byte
 - ▶ numpy: salvataggio di array
 - ▶ Immagini per poi elaborarle con opportune librerie tipo opencv
- ▶ File di input e output nel test di programmazione



Python – Apertura di un file

- ▶ Apertura sicura di un file nel caso di errori, il file viene chiuso automaticamente.

```
with open(filename, 'modalità') as file_descriptor:  
    operazione_sul_file(file_descriptor)
```

- ▶ Apertura non sicura: se l'operazione non ha successo e viene restituito un errore, il programma termina l'esecuzione ma il file rimane aperto

```
file_descriptor = open(filename, 'modalità')  
operazione_sul_file(file_descriptor)  
file_descriptor.close()
```

Python – Parametri

- ▶ Filename: nome del file (inserire anche il percorso se si trova in una cartella diversa da quella del codice)
- ▶ Modalità: quale sarà la funzione applicata al file (es: lettura scrittura)
- ▶ File_descriptor: variabile (oggetto) a cui viene associato il file aperto per eseguire l'operazione (modalità) scelta

Python - Modalità file

- ▶ Lettura: *r*
- ▶ Scrittura: *w*
- ▶ Append: *a*
- ▶ Creazione : *x*
- ▶ Testo: *t*
- ▶ Byte: *b*

```
with open("file.txt") as f:  
    l = f.readlines()
```

Es: leggere i byte di un file (rb), aggiungere del testo (at).

NB: r e t sono le modalità di default, si possono omettere

Python – Operazione sul file

▶ Lettura

```
content_file = fd.read() | content_file = fd.readlines()
```

#readlines ci restituisce una lista; read ci restituisce una stringa

```
fd.write(string_to_be_written) | fd.writelines(list_strings_to_be_written)
```

#writelines ci restituisce una lista; write ci restituisce una stringa

- ▶ Scrittura: cancella dati già presenti nel file
- ▶ Append: inserisce i nuovi dati alla fine del file

File csv

- ▶ Vengono usati per salvare tabelle e database su file
- ▶ Sono formati da righe di valori separati fra loro da un un carattere, detto separatore(solitamente una virgola)
- ▶ Vengono implementati principalmente attraverso il modulo "csv" o "pandas"

File csv: scrittura

```
import csv

def scrivi_csv(nome_file="dati.csv"):
    try:
        with open(nome_file, 'w', newline='', encoding='utf-8') as file_csv:
            writer = csv.writer(file_csv)
            writer.writerow(["nome", "cognome", "eta", "città"])
            writer.writerow(["Mario", "Rossi", "30", "Roma"])
            writer.writerow(["Luigi", "Verdi", "25", "Milano"])
            writer.writerow(["Anna", "Bianchi", "28", "Napoli"])
        print(f"File CSV '{nome_file}' scritto con successo.")
    except Exception as e:
        print(f"Errore durante la scrittura del file CSV: {e}")
```

File csv: lettura

```
def leggi_csv(nome_file="dati.csv"):
    try:
        with open(nome_file, 'r', encoding='utf-8') as file_csv:
            reader = csv.reader(file_csv)
            for riga in reader:
                print(riga)
    except FileNotFoundError:
        print(f"Errore: il file '{nome_file}' non è stato trovato.")
    except Exception as e:
        print(f"Errore durante la lettura del file CSV: {e}")
```


File json

- ▶ I file JSON vengono utilizzati in Python per memorizzare e scambiare dati strutturati in modo leggibile sia per gli esseri umani che per le macchine
- ▶ Principalmente vengono usati per memorizzare dati complessi e annidati, come ad esempio strutture di dati gerarchiche, elenchi di oggetti, o dizionari annidati.
- ▶ Implementati principalmente con il modulo "json"

File json: scrittura

```
import json

def scrivi_json(nome_file="dati.json"):
    dati = {
        "nome": "Mario",
        "cognome": "Rossi",
        "eta": 30,
        "citta": "Roma"
    }
    try:
        with open(nome_file, 'w', encoding='utf-8') as file_json:
            json.dump(dati, file_json, indent=4)
            print(f"File JSON '{nome_file}' scritto con successo.")
    except Exception as e:
        print(f"Errore durante la scrittura del file JSON: {e}")
```

File json: lettura

```
import json

def leggi_json(nome_file="dati.json"):
    try:
        with open(nome_file, 'r', encoding='utf-8') as file_json:
            dati = json.load(file_json)
            print("Dati letti dal file JSON:")
            print(dati)
    except FileNotFoundError:
        print(f"Errore: il file '{nome_file}' non è stato trovato.")
    except Exception as e:
        print(f"Errore durante la lettura del file JSON: {e}")
```

Esercizi

Python – Esercizio

- ▶ Scrivere su un file il vostro nome
- ▶ Leggere il contenuto del file
- ▶ Aggiungere una riga contenente l'età

Python – Args

- ▶ Permette di creare delle funzioni che accettano in input un numero non definito di parametri
- ▶ *args: riceve più argomenti come una tupla i quali però non hanno un nome definito

Python

```
def my_sum_args(*args):  
    """Somma un numero variabile di argomenti."""  
    total = 0  
    for num in args:  
        total += num  
    return total  
  
print(my_sum_args(1, 2, 3))          # Output: 6  
print(my_sum_args(10, 20, 30, 40)) # Output: 100  
print(my_sum_args())                 # Output: 0
```

Python – Kwargs

- ▶ Permette di creare delle funzioni che accettano in input un numero non definito di parametri
- ▶ ****kwargs**: riceve più argomenti come un dizionario i quali però hanno un nome definito

Python

```
# concatenate.py
def concatenate(**kwargs):
    result = ""
    # Iterating over the Python kwargs dictionary
    for arg in kwargs.values():
        result += arg
    return result

print(concatenate(a="Real", b="Python", c="Is", d="Great", e="!"))
```

Esercizi

Python – Esercizio

- ▶ Scrivere una funzione che prenda in input un numero indefinito di parametri di tipo stringa in minuscolo e ne restituisca una lista contenente gli stessi parametri ma in maiuscolo
 - ▶ Farlo sia con `*args` che con `**kwargs`
 - ▶ Ricordatevi che `kwargs` accetta parametri definiti come un dizionario

Python – Esercizio

- ▶ Scrivere una funzione che prenda in input un numero indefinito di parametri di tipo int e ne restituisca la somma
 - ▶ Farlo sia con `*args` che con `**kwargs`

Python – Esercizio

- ▶ Scrivere una funzione che prenda in input un numero indefinito di parametri di tipo stringa e ne restituisca una lista contenente gli stessi parametri
- ▶ Scrivere una funzione che prenda in input un numero indefinito di coppie <chiave:valore> e ne restituisca un dizionario

Python – Esercizio

- ▶ Scrivere un csv di 4 righe dove ogni riga contiene:

[nome,cognome,eta,matricola]

- ▶ Leggere solo le righe dispari

Python – Esercizio

- Scrivi una funzione Python che accetta un dizionario come input. La funzione deve salvare il dizionario in un file JSON.
- Scrivi un'altra funzione che legge il file JSON creato nel passaggio precedente e restituisce il dizionario Python originale.

Python – Esercizio

- Crea una funzione che accetta sia `*args` che `**kwargs`.
- La funzione deve stampare tutti gli argomenti posizionali (`*args`) e poi tutte le coppie chiave-valore (`**kwargs`).

Python – Esercizio

- Definisci una classe Libro con attributi come titolo, autore, e pagine.
- Implementa i metodi speciali `__str__` e `__repr__` per fornire rappresentazioni stringhe personalizzate degli oggetti Libro.

Python – Esercizio

- Scrivi alcune frasi in un file di testo, dove ogni frase è su una riga diversa.
- Leggi il contenuto del file riga per riga.
- Conta il numero totale di parole presenti nel file e stampa il conteggio.

Python – Esercizio

- Definisci una funzione Python, **processa_input**, che accetta sia argomenti posizionali (*args) che a parole chiave (**kwargs).
- La funzione deve:
- Stampare il numero di argomenti in *args
- Filtrare **kwargs per mantenere solo le coppie chiave-valore dove il valore è una stringa, e restituire il dizionario risultante.